BALANCING A BALL ON A BEAM

DESIGN PRESENTATION

KULIRANIYIL SAJU JESTIN

CHEKKATTU NANDAN NIKHIL

MADHU AMAL

SURESH AKHILESH

Introduction

Balancing a ball on a beam using an ESP32 microcontroller is a classic control systems project that demonstrates real-time feedback and actuation

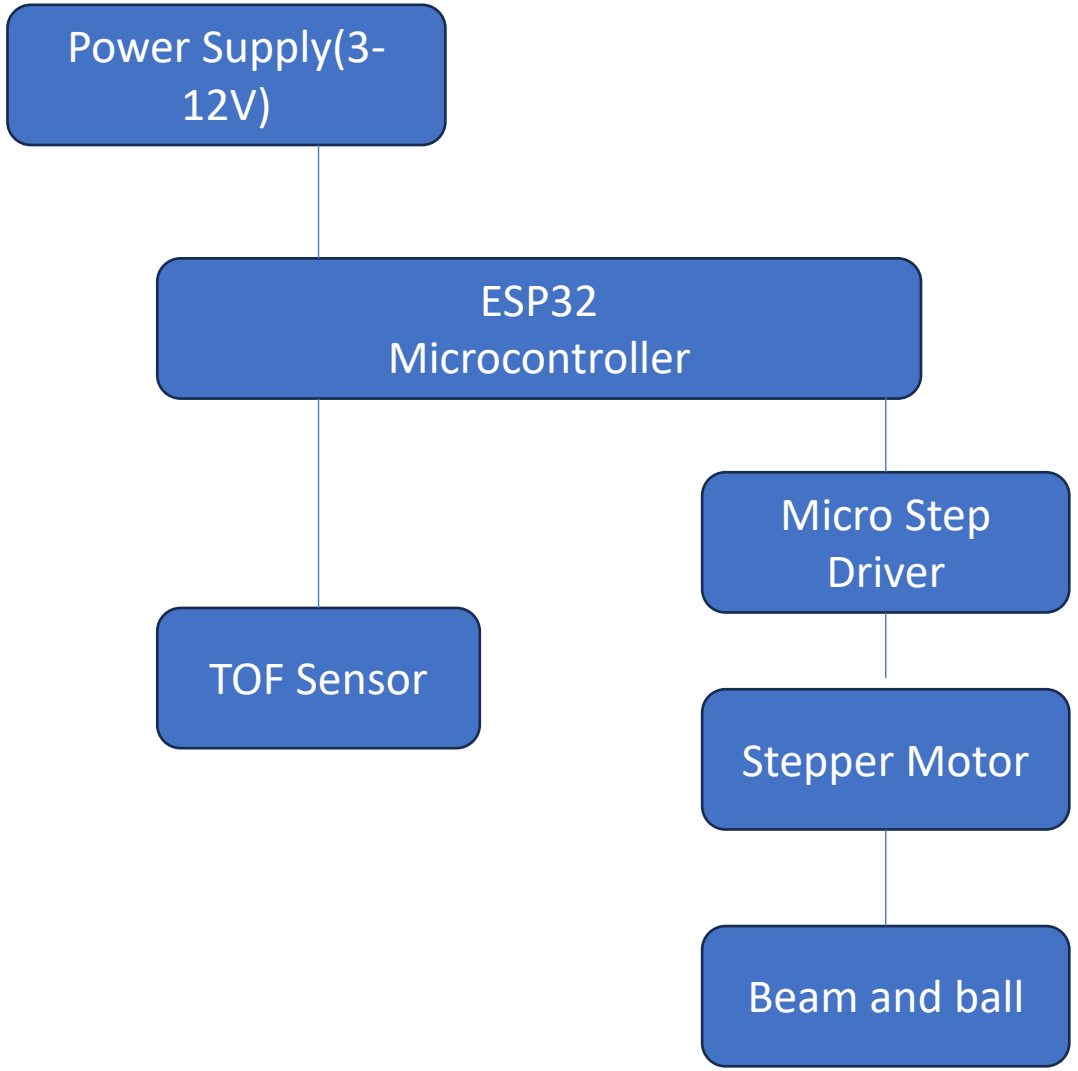
Objective: To balance a ball on a beam by controlling its position in real-time.

Components: ESP32 microcontroller, TOF sensor, Stepper motor, Power supply 3-12V, USB power supply 4C, Micro step driver, Beam(frame), Ball

Applications: Demonstrates control principles, real-time sensing, and microcontroller programming.

System Architecture

Block Diagram



System Architecture

Micro-Step Driver

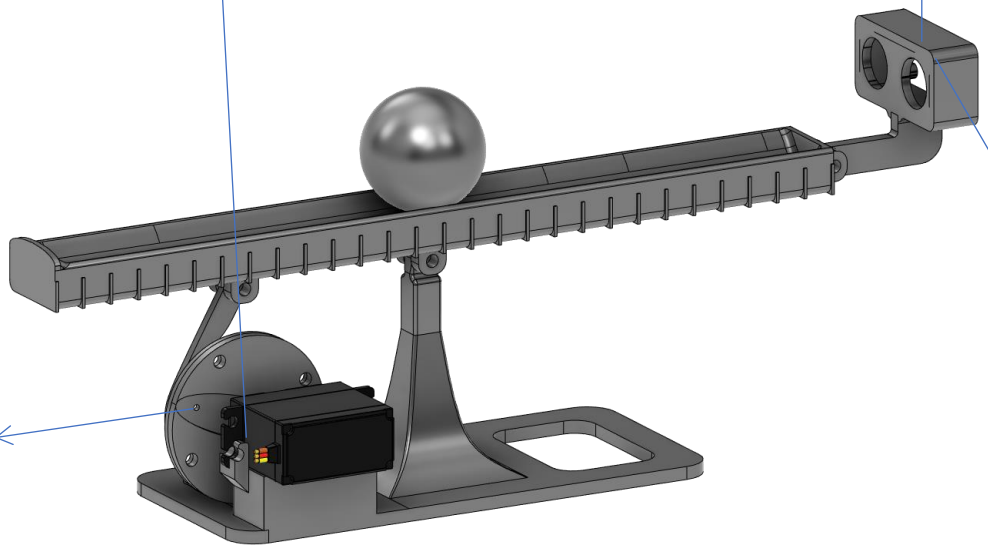


ESP 32 Micro-Controller





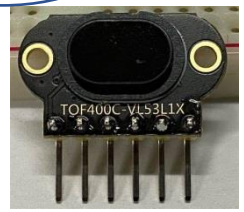
Power Supply



Ball on the beam assembly

Stepper Motor

ToF Sensors



Data Flow Between the components

1. **TOF(Time of Flight) Sensor to ESP32**

- The TOF sensor continuously sensos the data about the position of the ball on the beam and transfer the data in the form of digital signal, into the ESP32 micro controller providing real time feedback and on the position of ball

2. **ESP32 Micro controller**

- When the ESP32 micro controller received the data from the TOF sensor, it runs a controlled algorithm typically with PID controller to determine the necessary movement required for balancing the ball in the precise position
- The ESP32 micro controller send the data into the stepper motor, based on the computed control output

3. **Micro Step Driver to Stepper Motor**

- When the Micro step driver receive the signal from the ESP32 micro controller, it coverts the signal into precise motor movements. This driver dictate the step and direction the stepper motor should move and it make ensures fine motion and smooth controlling of the stepper motor which leads to high resolution positioning

4. Stepper Motor to beam:

- The stepper motor rotates in-order to tilt the beam, driven by the micro step driver. The movement of the beam results in the movement of ball.
- The motors adjustment result in keeping the ball in the precise position with respect to the output command from the ESP32

5. Power Supply:

- The whole system is powered up by a 3-12V via a USB power supply (4C) which is connected to the EPS32 via a USB A to Micro USB cable. From the micro controller the power is supplied to all other components via wire.

Components and Specifications

ESP32 Micro controller:

- CPU : Dual- core Xtensa LX^ processor, up to 240 MHz
- Ram: 520 KB SRAM
- I/O Pins:

✓ Digital I/O: Up to 34 GPIO pins

✓ Analog Input: Multiple ADC channel(12-bit)

- Timers and PWM:

✓ For precision and timing there are Multiple hardware timers

✓ For motor and servo control there is PWM support

- Power management:

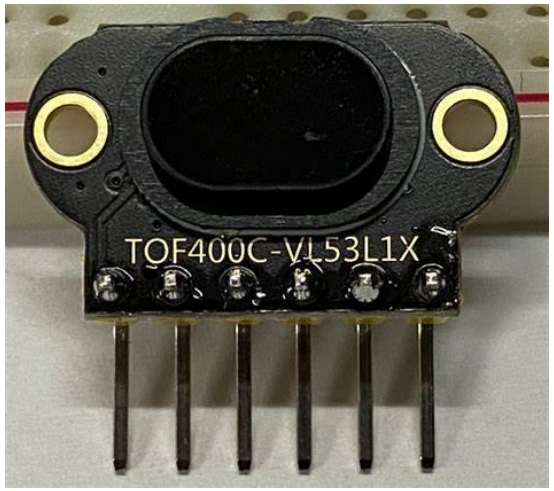
✓ Low-power sleep modes and deep sleep modes

✓ Operating voltage : 3.3V



Time-of-Flight(TOF) sensor:

The TOF sensor is used for measuring the distance using the principle of time of flight to calculate how long it takes for the light pulse to travel from the sensor to object and back



- Measurement range : 2-4m
- Accuracy : ±3% or better, which will depend on the distance and lightning conditions
- Field of view : 15° to 25°
- Measurement Time : 30ms to few 100 milliseconds, Very fast sampling rate.
- Operating voltage : 2.6V – 3.5V.

Micro step Driver:

It is used to control the stepper motor by dividing each step into small steps which results in the smooth and precise movement

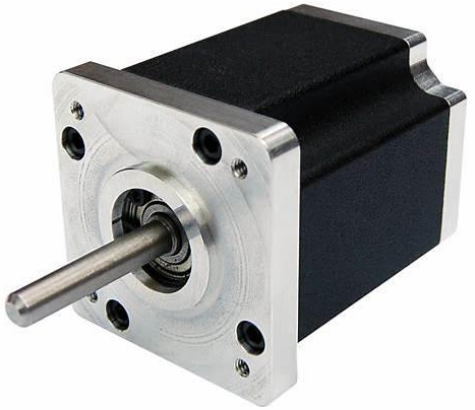
- Operating Voltage: Typically 9-42V similar to DM542.
- Current Rating: Adjustable current limit, usually up to 2.2A per phase with sufficient cooling .
- Micro stepping: Support for up to 1/32 micro stepping or 1/16 micro stepping (A4988).



Stepper Motor:

It is electro-mechanical device that converts electrical signals pulses into precise mechanical movements.

- Step angle: 1.8° per full revolution, that is equivalent to 200 steps per revolutions
- Rated current – 0.4A per phase.
- Holding torque: High torque is provided in-order to keep the beam steady



Power supply:

A reliable power supply is essential for ensuring that all the components operates accurately.

- Voltage range : 3V – 12V
- Current capacity : 2A – 5A
- USB power supply (4C) : Provides flexible option for powering the micro controller or other USB – compatible components, often rated at 5V with at least 2A output.

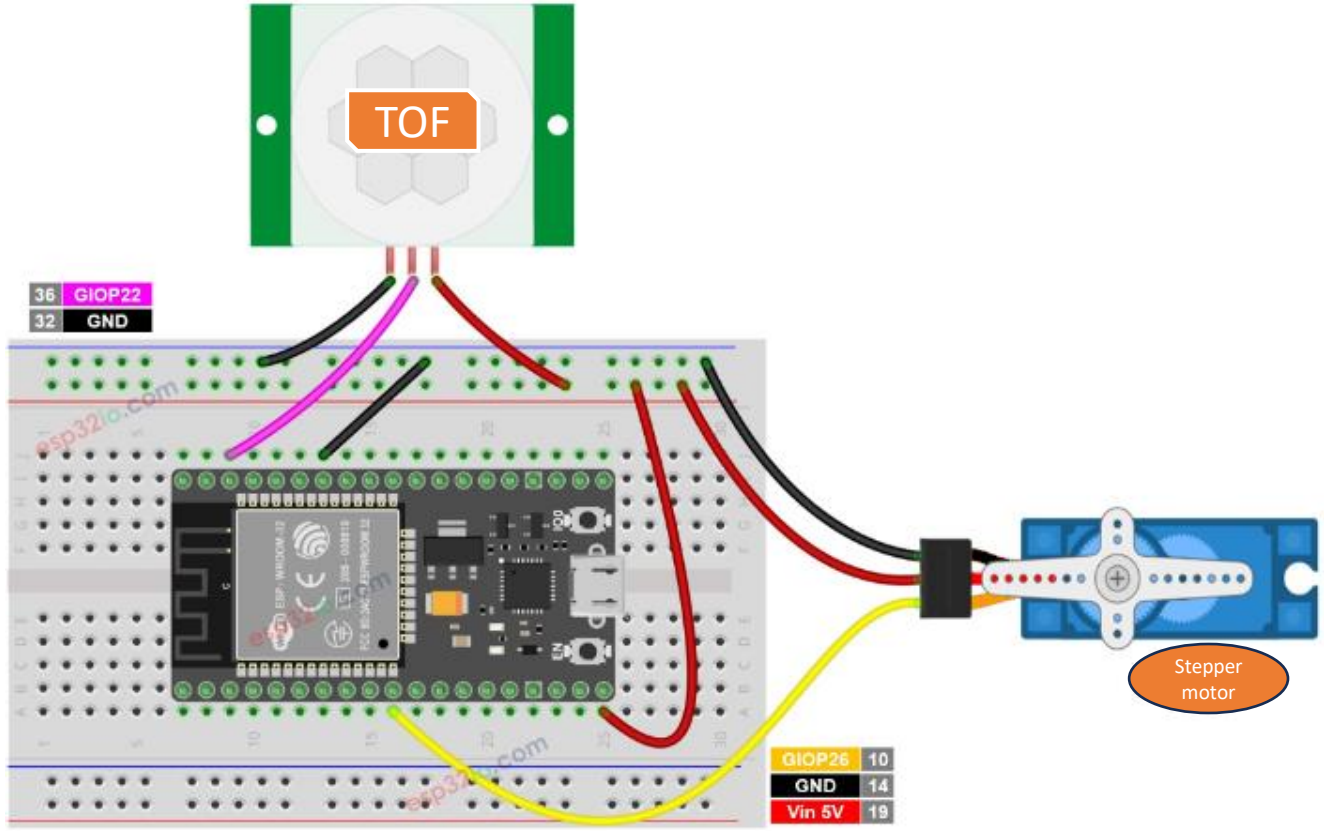
Mechanical components:

Beam : The beam we are using here is have a length of 48cm and material of the beam is stainless steel.

Ball : The ball we are using here is a table tennis ball with radius 20mm and the material of the ball is plastic and its weight is 2.7g

System Design & Schematic

Wire diagram connecting the ESP32, TOF sensor and stepper motor



Control System Design

Feedback loop

PID Controller : A PID(Proportional-Integral-Derivative) controller is a feedback system used to maintain the setpoint in a dynamic system by minimizing the error between the setpoint and actual outputs
In ball balancing on a beam the PID controller is used to adjust the stepper motor's position to control the angle of the beam and to keep the ball balanced

Working of a PID controller:

- Proportional (P): This component produce an output proportional to the current error(difference between the desired ball position and its actual position) . It provides an immediate correction based on the error.
- Integral (I): This part accumulates past error over time and helps eliminate any steady-state error by applying a correction based on the historical sum of these errors.
- Derivative (D) : This components reacts to the rate of change of error, offering a damping effect that helps reduce overshoot and stabilize the system by predicting the future behaviour based on the current trend.

Design considerations for tuning the PID controller

It involves selecting the appropriate P, I and D parameters to achieve optimal control.

1. Set Kp : Start with only Kp active and set Ki and Kd to zero. Increase Kp gradually until the system responds adequately but does not oscillate excessively.
2. Add Ki : Introduce Ki to eliminate any steady state error. Adjust Ki until stady state accuracy improves without causing instability.
3. Adjust Kd : Fine tune Kd to dampen oscillation and reduce overshoot. This helps the system settle quickly without excessive fluctuations.

Setpoint control: The desired position of the ball is the exact centre of the beam.

Mechanical design

- The ball is just placed on the beam to move freely over the beam.
- The beam is attached to the stepper motor via three connecting rods and in between the connecting rods. Pivot joints are used which allows free rotation in between and also allows smooth tilt of the beam. The joint is made up of plastic to reduce the weight
- One end of the beam is attached to the base plate via an other connecting rod and a pivot joint.
- The two TOF sensors are attached to the two ends of the beam
- The stepper motor and micro step driver are attached to the base plate.

Software design:

Here we are using the Arduino code and C++ language.

1. **Control Algorithm Implementation (PID Algorithm on The ESP32)**

The PID controller uses the following equation to compute the control output:

$$u(t) = K_p e(t) + K_I \int e(t)\,dt + K_D \frac{de(t)}{dt}$$

Where,

- ✓ u(t) is the control output (signal sent to the stepper motor).
- ✓ e(t) is the current error(difference between the set point and actual point).
- ✓ Kp, Ki and Kd are the proportional , integral and derivative gain respectively.

- Implementation on ESP32

- ➢ Uses an interrupt service routine(ISR) to ensure the sensor data is read at a consistent interval
- ➢ The PID calculation runs in the main control loop, processing the error and adjusting the output signal accordingly.

2. Data Acquisition Process from the TOF Sensor and Signal Processing

- TOF Sensor Data Acquisition:
  - The ESP32 reads the distance data from the TOF sensor using the I2C communication protocol.
  - The sensor continuously measures the position of the ball on the beam, sending the data to the ESP32 at a fixed interval.

- Signal Processing:
  - Implement basic filtering to reduce noise (e.g., moving average filter).
  - Ensure that the readings are accurate and free from significant fluctuations that could impact control stability.

3. Motor control code for precise movement based on PID output

- Motor control strategy:
  - The output from the PID controller is converted into pulse signals for the micro step driver to adjust the stepper motor.
  - The ESP32 generates the STEP and DIR signals based on the magnitude and sign of the PID output.
  - A positive output indicates moving the motor in one direction, while a negative output indicates the opposite direction.

Challenges and Solutions

1. Sensor Noise and inaccuracies

- The TOF sensor might produce noisy or error in distance measurements due environmental factors like ambient light, surface reflectivity of electrical reflectivity.
- To avoid this we can add a physical shielding around the sensor and we can use digital filters like Kalman filter to smooth out sensor data. Regular calibration of the sensor can be done to reduce the environmental influences.

2. Delay in Motor response

- Stepper motors and micro step drivers may show response delays, which will result in the system's ability to react quickly to changes in the ball's position.
- To avoid this we can use a high-torque stepper motor and a driver with higher current capabilities for faster and more precise movements.

3. Power supply issue

- Fluctuation in the power supply can affect the operation of the entire system
- To avoid this we can use a stabilized power supply and a battery backup to ensure consistent voltage and current supply and proper grounding can be provided to make ensure that all components share a common ground to avoid electrical noise.

Future Improvements

- Mechanical components can be redesigned of better controlling like the less weight materials like carbon can be used to built the beam
- Other sensors like accelerometer, gyroscopes can be used for providing better feedback.
- Add thermal sensors to monitor the stepper motor and driver temperatures and activate cooling or stop the system if overheating is detected.

Conclusion

In conclusion, the Ball Balancing on a Beam project utilizing an ESP32 microcontroller, TOF sensor, stepper motor, and other components demonstrates a successful integration of precise control and real-time processing. The system showcases the effective use of PID control to achieve dynamic stability, maintaining the ball's position on the beam through continuous sensor feedback and precise motor adjustments. Challenges such as sensor noise, motor response delays, have been addressed with robust strategies, ensuring reliable performance.